

The key insight is that Nityam is **not** a collection of AI tutoring features. It is one system with a persistent learning context, with different interaction modes sitting on top of it.

Nityam — MVP Definition

1. The core product

Your classroom becomes your personal AI tutor.

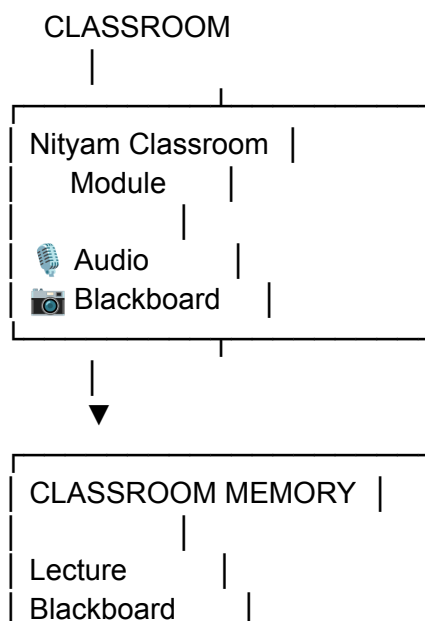
Nityam observes the classroom once through a dedicated classroom hardware module:

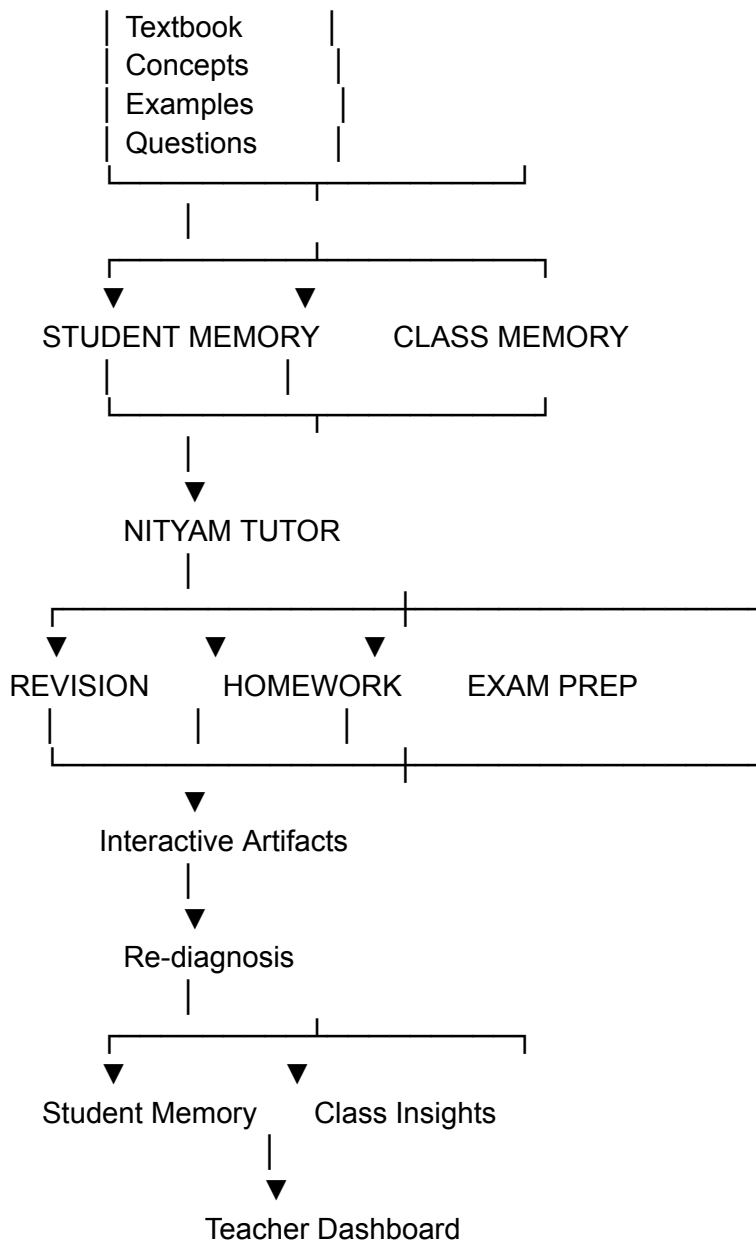
Microphone + blackboard-facing camera

It then reconstructs the lesson:

- what the teacher said
- what was written on the board
- relevant textbook knowledge
- examples used
- questions asked
- student interactions
- concepts taught
- misconceptions observed

After school, the student opens Nityam and chooses what they want to do.





This is essentially the product version of the classroom capture → diagnose → teach → re-diagnose loop already present in your deck.

2. The crucial architectural principle

I would make **one decision that governs almost everything we build**:

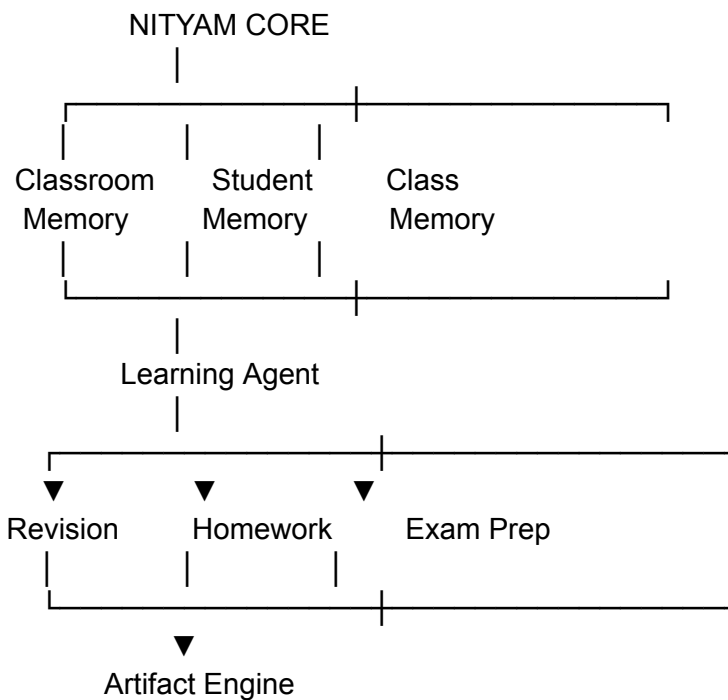
Modes are interfaces. The intelligence underneath them is shared.

So don't build:

Revision Agent
Homework Agent
Exam Agent
Doubt Agent

as four independent systems.

Instead:



This is what allows us to scale to other subjects later.

Physics is content.

The **agent architecture is Nityam.**

3. The student's home screen

Your proposed interface is right.

I would make the first version roughly:

NITYAM

Arjun 

Good evening, Arjun.

Here's what happened in class today.

PHYSICS

Projectile Motion

Today's class

Concepts covered: 6

Your current mastery: 72%

[Continue Learning]

What do you want to do?

Revision

Doubts

Homework

Exam Preparation

Your weak areas

- Horizontal velocity
- Resolving velocity into components

And yes, we should iterate this UI based on feedback.

4. Mode 1 — Revision

The student selects:

Revision

Then:

How would you like to revise?

Quick

A fast overview of today's class.

Standard

Understand + practice the important concepts.

Deep

Interactive, detailed revision with extensive practice.

Quick

The tutor leads almost entirely.

Tutor explains



Visual artifact



Next concept



Visual artifact



Short checkpoint



Next concept

Minimal questioning.

Standard

Explain



Visual



Check understanding



Student responds



Adapt



Next concept

Deep

Highly interactive:

Explain



Question



Student reasoning



Diagnose



Hint



Artifact



Student manipulates artifact



Follow-up question



Re-test

This makes "intensity" an actual behavioral parameter of the agent rather than merely a UI preference.

5. Mode 2 — Doubt Solving

This should be **student-led**.

Student:

"Why does horizontal velocity remain constant?"

Nityam should not immediately answer.

Agent behavior

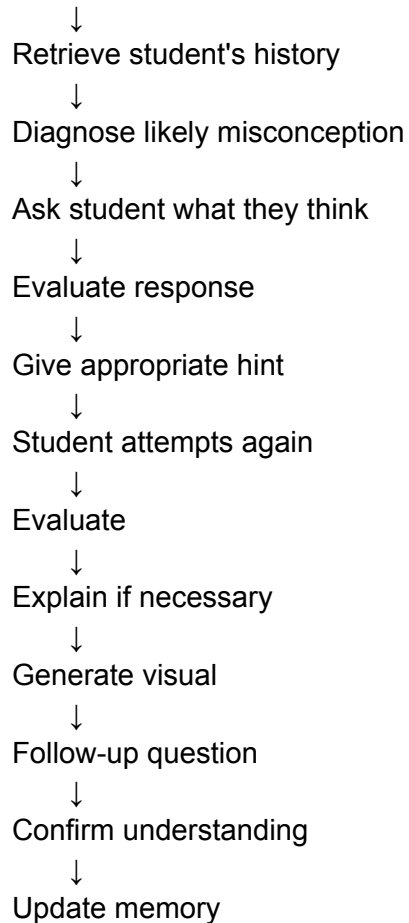
Student question



Understand question



Retrieve classroom context



The important part:

Nityam decides how much help to give.

That is much better than hardcoding:

"Always give three hints."

The agent can recognize:

Student understands the concept but made an arithmetic mistake.

versus:

Student fundamentally misunderstands the concept.

Those should trigger different interventions.

6. Mode 3 — Guided Homework

Teacher uploads homework.

The homework becomes associated with:

Class

+

Subject

+

Topic

+

Classroom context

+

Student profile

Then:

Student opens Question 4

Nityam sees:

Calculate the maximum height of the projectile.

Student attempts.

Nityam detects:

Formula selected correctly, but student has confused initial vertical velocity with total initial velocity.

Instead of saying:

"Wrong. Use $u \sin \theta$."

it might say:

"Before we continue, what component of the initial velocity actually determines how high the projectile goes?"

Then it generates an appropriate artifact.

Critical rule

Nityam teaches the student to solve the problem. It does not solve the homework for them.

And the agent determines when the student has genuinely exhausted their current reasoning before increasing assistance.

7. Mode 4 — Exam Preparation

This is not:

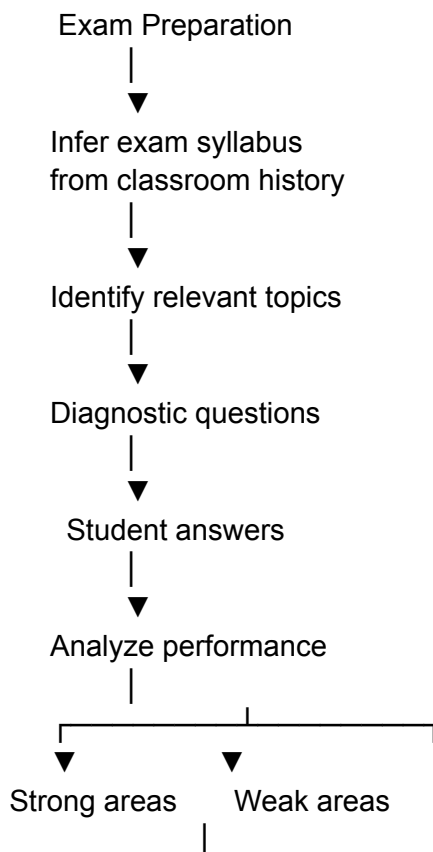
"Take a mock exam."

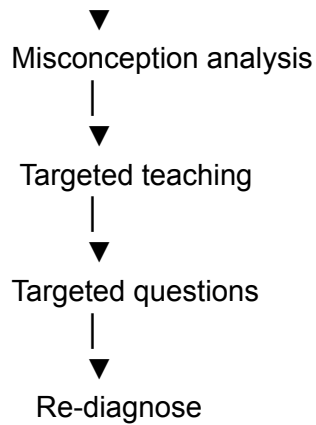
It is:

"Find out what I'm going to get wrong before my exam."

This is an important distinction.

The workflow:





At the end:

Your Exam Readiness

Projectile Motion	91%
Newton's Laws	84%
Work & Energy	73%
Circular Motion	51% ← Needs work

Main issue:

Confusion between centripetal acceleration and tangential acceleration.

Recommended:

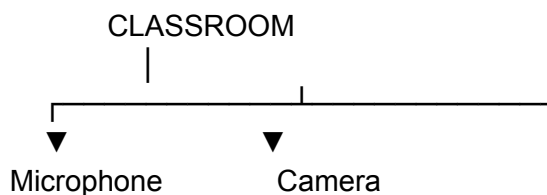
12-minute deep revision
+ 4 targeted problems

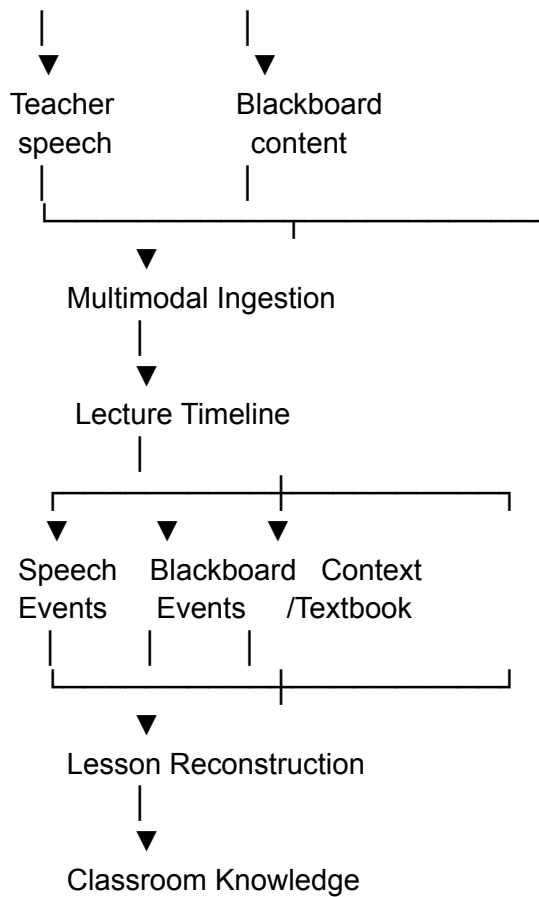
This is a much more compelling feature than generic exam generation.

8. Classroom ingestion

This is the foundation.

Your intended pipeline is correct:

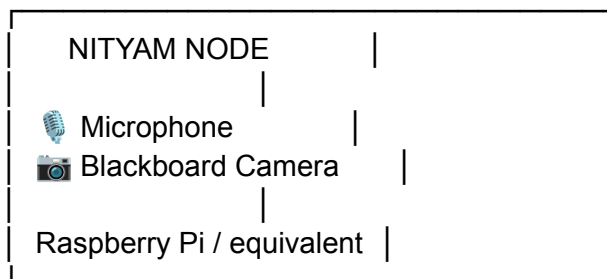




The hardware module

For MVP:

One classroom → one module



The camera is **blackboard-facing**, not primarily student-facing.

That is a very important scope decision.

9. Student-specific diagnosis starts automatically

You confirmed that Nityam should automatically analyze the student even before they explicitly interact with it.

So after ingestion:

Classroom



Lesson reconstructed



Concepts identified



Student interaction/evidence



Individual student model

For example:

ARJUN

Projectile Motion

Concept	Mastery
Trajectory	91%
Horizontal velocity	84%
Vertical velocity	77%
Time of flight	73%
Maximum height	62%
Maximum range	41% ←

Possible misconception:

Confuses total velocity with horizontal component.

Evidence:

3 interactions

This is much more valuable than simply producing:

"Here's a summary of today's class."

10. Class intelligence

At the same time:

CLASS 11A

Projectile Motion

Strongest concept:

Trajectory

Weakest concept:

Vector decomposition

Common misconception:

Students confuse u with u_x

Students needing intervention:

7 / 42

Recommended next-class clarification:

Spend 5 minutes revisiting
horizontal/vertical components.

And importantly, **Nityam suggests this to the teacher rather than automatically changing the teacher's lesson.**

That respects the teacher as the authority.

11. Teacher dashboard

I would make the MVP dashboard deliberately small.

Screen 1 — Today's Class

Class 11A

Physics

Projectile Motion

Class Understanding
72%

6 concepts taught

Concept	Mastery
Trajectory	91%
Time of Flight	83%
Maximum Height	76%
Vector Components	61%
Maximum Range	53%

Screen 2 — Intervention

Students needing attention

Arjun

→ Maximum range

→ Active misconception

Riya

→ Vector components

→ Weak conceptual understanding

Kabir

→ Sign convention

→ Repeated error

Screen 3 — Class Insights

Nityam noticed:

7 students struggled with
horizontal/vertical components.

Suggested clarification for next class:

Explain independence of x and y motion
using the same projectile example.

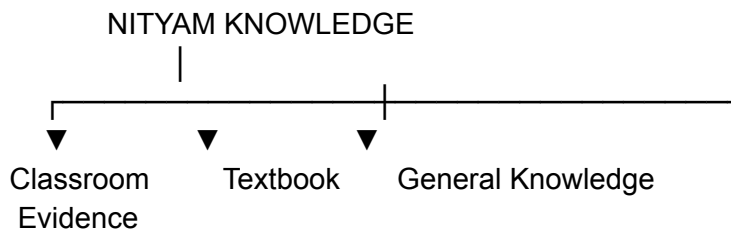
That directly extends the teacher-side product vision in your deck, where Nityam surfaces individual gaps and recommends intervention rather than replacing teaching.

12. Textbook grounding

This is also locked:

Teachers choose the textbook/content source from Nityam's library.

So we have three knowledge sources:



But they have different priorities.

My proposed priority

1. Classroom
2. Selected textbook
3. General knowledge

However, the agent **doesn't tell the student "your teacher was wrong."**

Instead:

"In today's class, we used approach A."

"The textbook presents the same concept using approach B."

"Both approaches arrive at the same result."

Or, if there's a genuine factual conflict, the teacher gets the flag.

This is a very important product principle:

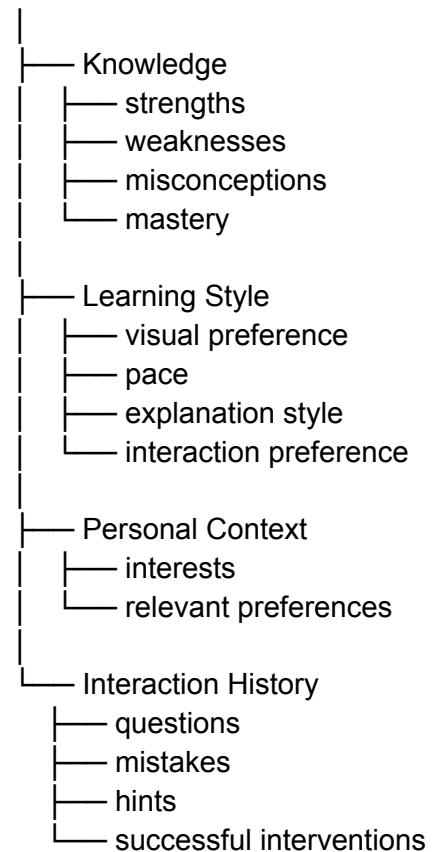
Nityam respects classroom continuity without becoming epistemically blind.

13. Memory architecture

I would make the memory layer one of our most important technical components.

Student memory

Student



And yes, interests can affect explanations.

If Arjun loves cricket:

Projectile motion → cricket ball trajectory.

If another student loves gaming:

Projectile motion → game physics.

But we should **not make this gimmicky**. The system should only use interests when they genuinely improve understanding.

14. Memory evolves

This is important.

We shouldn't store:

Weak at projectile motion

forever.

Instead:

Concept



Evidence



Hypothesis



Confidence



Interventions



New evidence



Updated belief

For example:

MISCONCEPTION

Horizontal velocity decreases
during projectile motion

Confidence: 82%

Evidence:

4 incorrect responses

Intervention:

Projectile simulator

After intervention:

3 correct responses

Current confidence:

27%

Status:
Likely resolved

That is a genuine **student model**, rather than a static profile.

15. Artifact architecture

I agree with your **5–8 polished primitives**.

For Physics MVP, I'd build exactly 7:

Primitive	Example
Interactive Graph	Position/velocity/time
Physics Simulator	Projectile motion
Interactive Diagram	Forces/vectors
Step-by-Step Derivation	Equations
Equation Manipulator	Rearranging equations
Interactive Question	MCQ / numerical / conceptual
Concept Visualizer	Dynamic conceptual animation

The agent does **not** invent arbitrary HTML every time.

Instead:

Student question



Agent determines pedagogical requirement



"Need projectile simulation"



Artifact Registry



ProjectileSimulator



Gemini supplies parameters

↓
Personalized artifact

And if the artifact already exists:

Existing artifact

↓
Retrieve

↓
Personalize

↓
Reuse

This is much faster, cheaper and more reliable.

16. Personalization of artifacts

Your Spider-Man example is actually useful.

Suppose two students learn projectile motion.

Student A likes Spider-Man.

Artifact:

Spider-Man throws a web at 45° .

Student B likes cricket.

Artifact:

Batter hits a cricket ball at 45° .

Same underlying simulator:

ProjectileSimulator

Different:

character
context
labels
narrative

examples

So:

The artifact is reusable. The pedagogical context is personalized.

That's an excellent architecture for scaling.

17. What the agent actually controls

This is where I want the hackathon version to become genuinely agentic.

The agent shouldn't merely generate answers.

It should decide:

What does the student need?

Student asks question



What is the goal?



What does the student already know?



What was taught in class?



What misconception is likely?



What teaching strategy should I use?



Do I ask a question?



Do I give a hint?



Do I explain?



Do I generate an artifact?



Do I reuse an artifact?



Did the student understand?



Update memory

That is the **Nityam Learning Agent**.

18. The agent's tools

Eventually I'd expose tools along these lines:

```
get_classroom_context()
get_lesson_timeline()
get_blackboard_content()
search_textbook()
get_student_profile()
get_student_history()
get_class_profile()

diagnose_concept()
generate_question()
evaluate_student_response()

get_artifact_template()
create_artifact()
update_artifact()

update_student_memory()
update_class_memory()

generate_student_report()
generate_teacher_insight()
flag_teacher_issue()
```

Then the LLM is actually **orchestrating a learning system**.

This will also make the Google ADK architecture much more meaningful than simply calling Gemini from a web app.

19. The most important hackathon demo

I think we've now arrived at a very strong 4-minute story.

Scene 1 — Classroom

20–30 sec

Teacher teaches:

Class 11 Physics — Projectile Motion

Camera watches the board.

Microphone captures the lecture.

We visibly show:

Nityam Classroom Node



Capturing

Scene 2 — Nityam processes the class

20 sec

Teacher leaves.

Nityam autonomously processes:

Audio

+

Blackboard

+

Textbook



Lesson reconstruction



Concepts



Class insights



Student models

No teacher intervention.

This directly demonstrates autonomy.

Scene 3 — Student logs in

15 sec

Dashboard:

Today's Physics Class

Projectile Motion

Your mastery: 68%

Weak area: Maximum Range

Student chooses:

Deep Revision

Scene 4 — The killer interaction

60–90 sec

Nityam:

"Let's start with projectile motion. Before I explain maximum range, what do you think determines it?"

Student answers incorrectly.

Nityam recognizes the misconception.

It doesn't immediately dump the answer.

It generates:

Interactive Projectile Simulator

Student moves:

Angle = 30°

Projectile changes.

Student:

"Oh, so 45° gives maximum range?"

Nityam:

"You're close. Let's test that."

Student interacts.

Nityam asks another question.

Student gets it.

Memory updates.

This demonstrates:

- voice
- multimodality
- classroom grounding
- reasoning
- personalization
- artifact generation
- adaptive teaching
- memory

in one scene.

Scene 5 — Teacher view

30 sec

Teacher dashboard:

Class 11A — Projectile Motion

Nityam:

"7 students are struggling with vector decomposition."

"I recommend revisiting this for 5 minutes tomorrow."

Teacher sees individual students.

Maybe one student has:

Maximum Range — misconception likely resolved

because of the interaction we just watched.

Scene 6 — The final punch

10 sec

Show:

ONE CLASSROOM
↓
42 STUDENTS
↓
42 LEARNING PATHS

Then:

Your classroom becomes your personal AI tutor.

That's the product.

20. What we are NOT building

This is where we protect ourselves from scope explosion.

For the hackathon MVP:

We are NOT building

- parent app
- payments
- school ERP

- attendance
- live classroom tutoring
- mobile app
- multiple subjects in the demo
- multiple grades in the demo
- arbitrary HTML generation
- dozens of artifact types
- production-scale student management
- sophisticated computer vision beyond what the demo needs
- real 42-student deployment
- full-year historical learning data
- complete textbook ecosystem
- complicated teacher workflow

We ARE building

- **Physics**
- **one grade**
- **one classroom**
- **one teacher**
- **one classroom hardware module**
- **one or two polished classroom recordings**
- **student platform**
- **four modes**
- **persistent memory**
- **class memory**
- **teacher dashboard**
- **interactive artifacts**
- **voice tutor**
- **autonomous post-class processing**
- **Google Cloud deployment**
- **Google ADK/Gemini agent architecture**

That is the MVP.

21. What I would call the actual MVP

Not:

"AI tutoring platform"

Not:

"Classroom recording + AI"

Not:

"Multimodal educational chatbot"

I'd define it internally as:

Nityam: Classroom-Grounded Personal Learning Agent

Input

Classroom Audio

+

Blackboard Video

+

Selected Textbook

+

Student Profile

Processing

Classroom Reconstruction

+

Student Diagnosis

+

Class Memory

+

Persistent Student Memory

Output

Revision

Doubt Solving

Guided Homework

Exam Preparation

+

Interactive Learning Artifacts

+

Teacher Insights

22. And this is the key differentiator

Your answer to "Why not ChatGPT?" is now much sharper.

ChatGPT

Knows physics.

Nityam

Knows your classroom.

More precisely:

Nityam knows what your teacher taught, how they taught it, what was written on the board, what the textbook says, what you already understand, what you repeatedly get wrong, what your classmates are struggling with, and how you learn best.

And then it **acts on that context**.

That is the product.

Your existing deck already frames the distinction around grounding the tutor in the actual lesson rather than giving students a generic syllabus.

23. One thing I would change from your original vision

You originally described:

"AI talks to student + generates HTML."

I would now describe it as:

"Nityam is a multimodal learning agent that can speak, reason, ask questions, remember, and construct interactive learning environments around the student."

The **interactive artifact** isn't a gimmick.

It is the agent's equivalent of a teacher drawing something on the board.

Teacher says:

"Imagine this projectile."

Nityam actually **builds the projectile**.

Teacher says:

"Move the angle to 30 degrees."

Nityam's artifact changes.

That's the experiential learning layer that ordinary chat interfaces don't provide.

Next: we should build the actual technical specification

The next step should **not** be more product brainstorming.

We should now go one level deeper and define:

1. **Exact system architecture**
2. **Google ADK agent hierarchy**
3. **Every agent/tool**
4. **Firestore/database schema**
5. **Student memory schema**
6. **Classroom memory schema**
7. **Audio/video processing pipeline**
8. **Blackboard understanding pipeline**
9. **Textbook/RAG architecture**
10. **Artifact Registry + 7 primitives**
11. **Voice interaction architecture**
12. **How the agent decides ask → hint → explain**
13. **How student diagnosis works**
14. **Teacher dashboard data model**
15. **Hardware architecture**
16. **Exact MVP frontend screens**
17. **What is genuinely implemented vs simulated**
18. **Hackathon demo script**
19. **Build order by hour**
20. **What we cut if we fall behind**

And I would design all of that specifically around the hackathon's three judging dimensions: **operational utility (40%), architecture (30%), and demo/production readiness (30%).**

At this point, **we should freeze the product vision above and move into the technical architecture.**